Rust libp2p 이벤트 처리 흐름 문서화

개요

이 문서는 Rust libp2p에서 이벤트가 어떻게 생성되고, 전파되며, 처리되는지에 대한 상세한 분석을 제공합니다. libp2p의 이벤트 처리 시스템은 비동기 프로그래밍 모델을 기반으로 하며, 다양한 계층 간의 효율적인 통신을 가능하게 합니다.

이벤트 처리 아키텍처 개요

전체 이벤트 흐름

```
┌─────────────────────────────────────────────────────────────┐
│                        Application Layer                    │
│ ↑ ToSwarm Events                        ↓ Application Commands │
├─────────────────────────────────────────────────────────────┤
│                        NetworkBehaviour                     │
│ ↑ ToBehaviour Events                  ↓ FromBehaviour Events │
├─────────────────────────────────────────────────────────────┤
│                        ConnectionHandler                    │
│ ↑ Handler Events                          ↓ Handler Commands │
├─────────────────────────────────────────────────────────────┤
│                           Swarm Core                        │
│ ↑ Connection Events                     ↓ Connection Commands │
```

```
├──────────────────────────────────────────────────────────────┤
│                        Connection Pool                       │
│ ↑ Transport Events                      ↓ Transport Commands │
├──────────────────────────────────────────────────────────────┤
│                       Transport Layer                        │
│ ↑ Network Events                        ↓ Network Operations │
└──────────────────────────────────────────────────────────────┘
```

이벤트 타입 분류

1. **Network Events**: 네트워크 레벨 이벤트 (연결 설정/해제, 데이터 수신 등)
2. **Transport Events**: 전송 계층 이벤트 (주소 변경, 리스너 상태 등)
3. **Connection Events**: 연결 레벨 이벤트 (핸드셰이크 완료, 서브스트림 생성 등)
4. **Protocol Events**: 프로토콜별 이벤트 (메시지 수신, 쿼리 완료 등)
5. **Swarm Events**: 상위 레벨 이벤트 (피어 발견, 동작 변경 등)

핵심 이벤트 처리 컴포넌트

1. Swarm 이벤트 루프

SwarmEvent 구조

```rust
pub enum SwarmEvent<TBvEvent, THandlerErr> {
/// NetworkBehaviour에서 생성된 이벤트
Behaviour(TBvEvent),
```

```
/// 새로운 연결이 설정됨

ConnectionEstablished {

peer_id: PeerId,

connection_id: ConnectionId,

endpoint: ConnectedPoint,

num_established: NonZeroU32,

concurrent_dial_errors: Option<Vec<(Multiaddr, TransportError<io::Error>)>>,

established_in: Duration,

},

/// 연결이 종료됨

ConnectionClosed {

peer_id: PeerId,

connection_id: ConnectionId,

endpoint: ConnectedPoint,

num_established: u32,

cause: Option<ConnectionError<THandlerErr>>,

},

/// 새로운 리스너가 시작됨

NewListenAddr {

listener_id: ListenerId,

address: Multiaddr,

},

/// 외부 주소 후보 발견
```

```
NewExternalAddrCandidate {
address: Multiaddr,
},
/// 외부 주소 확인됨
ExternalAddrConfirmed {
address: Multiaddr,
},
/// 다이얼 시도 실패
OutgoingConnectionError {
peer_id: Option<PeerId>,
connection_id: ConnectionId,
error: DialError,
},
/// 인바운드 연결 오류
IncomingConnectionError {
connection_id: ConnectionId,
local_addr: Multiaddr,
send_back_addr: Multiaddr,
error: ListenError,
},
}
```

Swarm 폴링 메커니즘

```rust
impl<TBehaviour> Future for Swarm<TBehaviour>
where
TBehaviour: NetworkBehaviour,
{
type Output = SwarmEvent<TBehaviour::ToSwarm, THandlerErr<TBehaviour>>;


fn poll(mut self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output> {
loop {
// 1. NetworkBehaviour 폴링
match self.behaviour.poll(cx) {
Poll::Ready(behaviour_event) => {
return Poll::Ready(SwarmEvent::Behaviour(behaviour_event.event));
}
Poll::Pending => {}
}


// 2. Connection Pool 폴링
match self.pool.poll(cx) {
Poll::Ready(pool_event) => {
return Poll::Ready(self.handle_pool_event(pool_event));
}
Poll::Pending => {}
```

```
}

// 3. Transport 폴링
match self.transport.poll(cx) {
Poll::Ready(transport_event) => {
return Poll::Ready(self.handle_transport_event(transport_event));
}
Poll::Pending => {}
}

// 모든 컴포넌트가 Pending이면 대기
return Poll::Pending;
}
}
}
```

2. NetworkBehaviour 이벤트 처리

ToSwarm 이벤트 타입

```
pub enum ToSwarm<TOutEvent, THandlerInEvent> {
/// 애플리케이션에게 전달할 이벤트 생성
GenerateEvent(TOutEvent),
/// 특정 피어에게 다이얼 시도
```

```
Dial {
opts: DialOpts,
},
/// 특정 주소에서 리스닝 시작
ListenOn {
opts: ListenOpts,
},
/// 리스너 제거
RemoveListener {
id: ListenerId,
},
/// ConnectionHandler에게 이벤트 전달
NotifyHandler {
peer_id: PeerId,
handler: NotifyHandler,
event: THandlerInEvent,
},
/// 외부 주소 후보 보고
NewExternalAddrCandidate(Multiaddr),
/// 외부 주소 확인
ExternalAddrConfirmed(Multiaddr),
/// 외부 주소 만료
ExternalAddrExpired(Multiaddr),
```

```
/// 연결 종료
CloseConnection {
peer_id: PeerId,
connection: CloseConnection,
},
}
```

FromSwarm 이벤트 타입

```
pub enum FromSwarm<'a, THandlerInEvent, THandlerOutEvent> {
/// 새로운 연결 설정됨
ConnectionEstablished(ConnectionEstablished<'a>),
/// 연결 종료됨
ConnectionClosed(ConnectionClosed<'a>),
/// 주소 변경됨
AddressChange(AddressChange<'a>),
/// 다이얼 실패
DialFailure(DialFailure<'a>),
/// 새로운 리스너
NewListener(NewListener<'a>),
/// 새로운 리슨 주소
NewListenAddr(NewListenAddr<'a>),
/// 리슨 주소 만료
ExpiredListenAddr(ExpiredListenAddr<'a>),
```

```
/// 리스너 종료
ListenerClosed(ListenerClosed<'a>),
/// 리스너 오류
ListenerError(ListenerError<'a>),
/// 새로운 외부 주소 후보
NewExternalAddrCandidate(NewExternalAddrCandidate<'a>),
/// 외부 주소 확인됨
ExternalAddrConfirmed(ExternalAddrConfirmed<'a>),
/// 외부 주소 만료됨
ExternalAddrExpired(ExternalAddrExpired<'a>),
}
```

3. ConnectionHandler 이벤트 처리

ConnectionHandlerEvent 타입

```
pub enum ConnectionHandlerEvent<TOutboundOpenInfo, TCustom> {
/// 새로운 아웃바운드 서브스트림 요청
OutboundSubstreamRequest {
protocol: SubstreamProtocol<TOutboundOpenInfo>,
},
/// NetworkBehaviour에게 이벤트 전달
NotifyBehaviour(TCustom),
/// 원격 피어의 프로토콜 지원 정보 보고
```

```
ReportRemoteProtocols {
protocols: Vec<StreamProtocol>,
},
/// 연결 유지 여부 변경
Close(CloseReason),
}
```

ConnectionEvent 타입

```
pub enum ConnectionEvent<TInboundOpenInfo, TOutboundOpenInfo, TInboundOpenInfoSe
/// 인바운드 서브스트림 네고시에이션 완료
FullyNegotiatedInbound(FullyNegotiatedInbound<TInboundOpenInfo>),
/// 아웃바운드 서브스트림 네고시에이션 완료
FullyNegotiatedOutbound(FullyNegotiatedOutbound<TOutboundOpenInfo>),
/// 아웃바운드 업그레이드 오류
DialUpgradeError(DialUpgradeError<TOutboundOpenInfoSend>),
/// 인바운드 업그레이드 오류
ListenUpgradeError(ListenUpgradeError<TInboundOpenInfoSend>),
/// 주소 변경
AddressChange(AddressChange),
/// 로컬 프로토콜 변경
LocalProtocolsChange(ProtocolsChange<'static>),
/// 원격 프로토콜 변경
```

```
RemoteProtocolsChange(ProtocolsChange<'static>),

}
```

상세 이벤트 처리 흐름

1. 연결 설정 이벤트 흐름

```
1. Transport Layer

↓ 물리적 연결 설정

2. Security Handshake

↓ 보안 핸드셰이크 완료

3. Multiplexer Initialization

↓ 스트림 멀티플렉서 초기화

4. Connection Pool

↓ ConnectionEstablished 이벤트 생성

5. Swarm

↓ FromSwarm::ConnectionEstablished 전달

6. NetworkBehaviour

↓ handle_established_connection() 호출

7. ConnectionHandler 생성

↓ 핸들러 초기화 및 등록

8. Application
```

↓ SwarmEvent::ConnectionEstablished 전달

코드 예제: 연결 설정 처리

```
// Pool에서 연결 설정 이벤트 생성
fn handle_established_connection(&mut self, connection: EstablishedConnection) {
let peer_id = connection.peer_id();
let connection_id = connection.id();
// NetworkBehaviour에게 알림
let handler = self.behaviour.handle_established_inbound_connection(
connection_id,
peer_id,
&connection.local_addr(),
&connection.remote_addr(),
)?;
// 연결을 풀에 추가
self.established.insert(connection_id, connection);
// 이벤트 생성
self.pending_events.push_back(PoolEvent::ConnectionEstablished {
peer_id,
connection_id,
endpoint: connection.endpoint().clone(),
num_established: self.num_peer_connections(peer_id),
concurrent_dial_errors: None,
```

```
established_in: connection.established_in(),
});
}
```

2. 프로토콜 메시지 처리 흐름

```
1. Network I/O
↓ 네트워크에서 데이터 수신
2. Transport Decoding
↓ 전송 계층에서 데이터 디코딩
3. Security Decryption
↓ 보안 계층에서 복호화
4. Multiplexer Demuxing
↓ 멀티플렉서에서 서브스트림 분리
5. Protocol Upgrade
↓ 프로토콜별 업그레이드 처리
6. ConnectionHandler
↓ FullyNegotiatedInbound 이벤트 처리
7. Protocol Processing
↓ 프로토콜별 메시지 처리
8. NetworkBehaviour
↓ on_connection_handler_event() 호출
9. Application Event
```

```
↓ ToSwarm::GenerateEvent 생성

10. Application

↓ SwarmEvent::Behaviour 수신
```

코드 예제: 메시지 처리

```
// ConnectionHandler에서 메시지 수신 처리

fn on_connection_event(&mut self, event: ConnectionEvent<...>) {

match event {

ConnectionEvent::FullyNegotiatedInbound(FullyNegotiatedInbound {

protocol: stream,

info: (),

}) => {

// 프로토콜별 메시지 처리

let message = self.decode_message(stream).await?;

// NetworkBehaviour에게 이벤트 전달

self.pending_events.push_back(

ConnectionHandlerEvent::NotifyBehaviour(

HandlerEvent::MessageReceived { message }

)

);

}

_ => {}
```

```
}
}

// NetworkBehaviour에서 핸들러 이벤트 처리
fn on_connection_handler_event(
&mut self,
peer_id: PeerId,
connection_id: ConnectionId,
event: HandlerEvent,
) {
match event {
HandlerEvent::MessageReceived { message } => {
// 메시지 처리 로직
let response = self.process_message(peer_id, message);
// 애플리케이션에게 이벤트 전달
self.pending_events.push_back(ToSwarm::GenerateEvent(
BehaviourEvent::MessageProcessed { peer_id, response }
));
}
}
}
```

3. 아웃바운드 요청 처리 흐름

1. Application Request

↓ 애플리케이션에서 요청 시작

2. NetworkBehaviour

↓ 요청을 핸들러 이벤트로 변환

3. ToSwarm::NotifyHandler

↓ 특정 핸들러에게 이벤트 전달

4. ConnectionHandler

↓ on_behaviour_event() 처리

5. OutboundSubstreamRequest

↓ 새로운 서브스트림 요청

6. Protocol Negotiation

↓ multistream-select 네고시에이션

7. FullyNegotiatedOutbound

↓ 네고시에이션 완료 이벤트

8. Protocol Execution

↓ 실제 프로토콜 실행

9. Response Processing

↓ 응답 처리 및 이벤트 생성

10. Application Response

↓ 최종 결과를 애플리케이션에 전달

코드 예제: 아웃바운드 요청

```
// 애플리케이션에서 요청 시작

impl MyBehaviour {

pub fn send_request(&mut self, peer_id: PeerId, request: MyRequest) -> RequestId

let request_id = RequestId::new();

// 핸들러에게 요청 전달

self.pending_events.push_back(ToSwarm::NotifyHandler {

peer_id,

handler: NotifyHandler::Any,

event: HandlerInEvent::SendRequest { request_id, request },

});

request_id

}

}



// ConnectionHandler에서 요청 처리

fn on_behaviour_event(&mut self, event: HandlerInEvent) {

match event {

HandlerInEvent::SendRequest { request_id, request } => {

// 새로운 서브스트림 요청

self.pending_events.push_back(

ConnectionHandlerEvent::OutboundSubstreamRequest {

protocol: SubstreamProtocol::new(
```

```
MyProtocolUpgrade::new(request),
request_id,
),
}
);
}
}
}
```

이벤트 처리 최적화 기법

1. 이벤트 배치 처리

```
// 여러 이벤트를 한 번에 처리
fn poll(&mut self, cx: &mut Context<'_>) -> Poll<ToSwarm<Self::ToSwarm, THandler
// 배치로 이벤트 처리
let mut events_to_process = Vec::new();
// 큐에서 여러 이벤트를 한 번에 가져옴
while let Some(event) = self.pending_events.pop_front() {
events_to_process.push(event);
// 배치 크기 제한
if events_to_process.len() >= MAX_BATCH_SIZE {
break;
```

```
}

}

if !events_to_process.is_empty() {

// 배치 처리

let result = self.process_event_batch(events_to_process);

return Poll::Ready(result);

}

Poll::Pending

}
```

2. 이벤트 우선순위 처리

```
use std::collections::BinaryHeap;


struct PrioritizedEvent {

event: ToSwarm<MyEvent, MyHandlerEvent>,

priority: u8,

timestamp: Instant,

}


impl Ord for PrioritizedEvent {

fn cmp(&self, other: &Self) -> Ordering {

// 높은 우선순위가 먼저 처리됨
```

```
self.priority.cmp(&other.priority)
.then_with(|| other.timestamp.cmp(&self.timestamp))
}
}

struct MyBehaviour {
priority_queue: BinaryHeap<PrioritizedEvent>,
}

impl MyBehaviour {
fn add_high_priority_event(&mut self, event: ToSwarm<MyEvent, MyHandlerEvent>) {
self.priority_queue.push(PrioritizedEvent {
event,
priority: 10, // 높은 우선순위
timestamp: Instant::now(),
});
}
}
```

3. 백프레셔 관리

```
const MAX_PENDING_EVENTS: usize = 1000;
```

```
impl MyBehaviour {

fn add_event(&mut self, event: ToSwarm<MyEvent, MyHandlerEvent>) -> Result<(), E

if self.pending_events.len() >= MAX_PENDING_EVENTS {

// 큐가 가득 찬 경우 오래된 이벤트 제거 또는 오류 반환

return Err(EventQueueFull);

}

self.pending_events.push_back(event);

Ok(())

}

fn poll(&mut self, cx: &mut Context<'_>) -> Poll<ToSwarm<Self::ToSwarm, THandler

// 큐 크기 모니터링

if self.pending_events.len() > MAX_PENDING_EVENTS / 2 {

tracing::warn!("Event queue is getting full: {}", self.pending_events.len());

}

// 이벤트 처리...

}

}
```

오류 처리 및 복구

1. 이벤트 처리 오류

```
fn handle_event_error(&mut self, error: EventProcessingError) {
```

```
match error {
EventProcessingError::TemporaryFailure(e) => {
// 일시적 오류: 재시도 큐에 추가
self.retry_queue.push_back(RetryableEvent {
event: e.event,
retry_count: e.retry_count + 1,
next_retry: Instant::now() + Duration::from_secs(1),
});
}
EventProcessingError::PermanentFailure(e) => {
// 영구적 오류: 로깅 후 폐기
tracing::error!("Permanent event processing failure: {:?}", e);
// 오류 이벤트 생성
self.pending_events.push_back(ToSwarm::GenerateEvent(
MyEvent::EventProcessingFailed { error: e }
));
}
}
}
```

2. 연결 오류 처리

```
fn on_swarm_event(&mut self, event: FromSwarm) {
```

```
match event {
FromSwarm::ConnectionClosed(ConnectionClosed {
peer_id,
connection_id,
cause,
..
}) => {
// 연결 종료 시 관련 상태 정리
self.cleanup_connection_state(peer_id, connection_id);
// 오류 원인에 따른 처리
if let Some(ConnectionError::Handler(handler_error)) = cause {
self.handle_handler_error(peer_id, handler_error);
}
// 재연결 필요 여부 판단
if self.should_reconnect(peer_id) {
self.pending_events.push_back(ToSwarm::Dial {
opts: DialOpts::peer_id(peer_id).build(),
});
}
}
_ => {}
}
```

```
}
```

성능 모니터링 및 디버깅

1. 이벤트 메트릭 수집

```
use std::sync::atomic::{AtomicU64, Ordering};

struct EventMetrics {
events_processed: AtomicU64,
events_dropped: AtomicU64,
average_processing_time: AtomicU64,
queue_size_histogram: [AtomicU64; 10],
}

impl EventMetrics {
fn record_event_processed(&self, processing_time: Duration) {
self.events_processed.fetch_add(1, Ordering::Relaxed);
// 이동 평균 계산
let current_avg = self.average_processing_time.load(Ordering::Relaxed);
let new_avg = (current_avg * 9 + processing_time.as_nanos() as u64) / 10;
self.average_processing_time.store(new_avg, Ordering::Relaxed);
}
```

```
fn record_queue_size(&self, size: usize) {
let bucket = (size / 100).min(9);
self.queue_size_histogram[bucket].fetch_add(1, Ordering::Relaxed);
}
}
```

2. 이벤트 트레이싱

```
use tracing::{instrument, debug, warn};

impl MyBehaviour {
#[instrument(level = "debug", skip(self, event))]
fn process_event(&mut self, event: MyEvent) -> Result<(), ProcessingError> {
debug!("Processing event: {:?}", event);
let start_time = Instant::now();
let result = self.do_process_event(event);
let processing_time = start_time.elapsed();
if processing_time > Duration::from_millis(100) {
warn!("Slow event processing: {:?}", processing_time);
}
result
}
}
```

3. 이벤트 덤프 및 분석

```
#[cfg(debug_assertions)]
struct EventDumper {
events: VecDeque<(Instant, String)>,
max_events: usize,
}

#[cfg(debug_assertions)]
impl EventDumper {
fn record_event<T: fmt::Debug>(&mut self, event: &T) {
let event_str = format!("{:?}", event);
self.events.push_back((Instant::now(), event_str));
if self.events.len() > self.max_events {
self.events.pop_front();
}
}
fn dump_recent_events(&self) -> Vec<String> {
self.events.iter()
.map(|(timestamp, event)| format!("{:?}: {}", timestamp, event))
.collect()
}
}
```

모범 사례 및 권장사항

1. 이벤트 설계 원칙

- **단일 책임**: 각 이벤트는 하나의 명확한 목적을 가져야 함
- **불변성**: 이벤트 데이터는 불변으로 설계
- **직렬화 가능**: 필요시 네트워크 전송이나 저장이 가능하도록 설계
- **타입 안전성**: 강타입 시스템을 활용한 컴파일 타임 검증

2. 성능 최적화

- **이벤트 풀링**: 자주 사용되는 이벤트 객체 재사용
- **지연 평가**: 필요할 때까지 비용이 큰 연산 지연
- **배치 처리**: 여러 이벤트를 한 번에 처리
- **우선순위 큐**: 중요한 이벤트 우선 처리

3. 오류 처리

- **복구 가능성 구분**: 일시적 오류와 영구적 오류 구분
- **재시도 전략**: 지수 백오프를 사용한 재시도
- **회로 차단기**: 연속 실패 시 일시적 차단
- **우아한 성능 저하**: 부분적 기능 장애 시에도 핵심 기능 유지

4. 모니터링 및 관찰성

- **구조화된 로깅**: 검색 가능한 로그 형식 사용
- **메트릭 수집**: 처리량, 지연시간, 오류율 모니터링
- **분산 추적**: 이벤트 흐름 전체 추적

- **헬스 체크**: 시스템 상태 정기 점검

이 문서는 Rust libp2p의 이벤트 처리 시스템에 대한 포괄적인 이해를 제공하며, 실제 구현 시 참고할 수 있는 구체적인 예제와 모범 사례를 포함합니다.